AI Engineering and Claude Architect · Part III, Agents

AI Frameworks, Lesson 4

Spring AI and LangChain4j

The order database's password failure reached the model word for word, along with its IP address, because the tool threw an exception and that's what the framework's default does. Nobody had written code to send it. Nobody had written code to stop it either.

Module	AI07 of 13
Lesson	4 of 4
Prerequisites	AI03, AI06 L2–L3, L8, AI07 L1
Exam relevance	Out of scope. Forced `tool_choice` differences relate to TS 2.3
Reading time	~45 minutes

Contents

1. Spring AI and LangChain4j
2. Module Revision

Lesson 4 of 4 · AI Frameworks

Spring AI and LangChain4j

> **Key idea**
>
> **Both give Java teams annotated tools, a built-in loop, memory, structured output and RAG, in a few lines.** Spring AI fits Spring shops, with `ChatClient` and advisors, and LangChain4j is framework-neutral, with interfaces as services. **Their defaults need changing for production: tool exceptions reach the model unless you stop them, and you should test how each behaves on your exact model.**

> *Before: the loop, tools and hooks in AI06. Teaches: the core APIs of both, what they do by default, how to compare them, and when to use neither. Used by: AI08.*

The hook: real frameworks, scripted model

These labs run **real Spring AI 2.0.2 and LangChain4j 1.23.0** against a scripted `ChatModel`. No API key is involved. The model's replies are fixed, and what's real is everything the frameworks do around them.

The tool throws an exception holding a secret. What does the model receive? Real output:

```
Spring AI:
  ToolResponseMessage{responses=[ToolResponse[id=t1, name=lookupOrder,
  responseData=orders-db password rejected at 10.2.3.4]], ...}
LangChain4j:
  ToolExecutionResultMessage{id='t1', toolName='lookupOrder',
  contents=[TextContent { text = "orders-db password rejected at 10.2.3.4" }], isError=true, ...}
LangChain4j, with failInvocationUnlessVisibleToLlm:
  the call failed, and the model made 1 request(s)
```

> *Key idea*
>
> **Both send the raw exception message to the model by default.** That's documented. LangChain4j's docs warn that it "can expose sensitive data", and its recommended handler stopped it here. Sending errors back is a good default for recovery (AI06 L5), but only if your exception messages are safe to share.

Line breaks are added, and the output is trimmed with `...`.

Intuition: two ways to hire an assistant

LangChain4j gives you a job description. You write an interface saying what you want, and it staffs the role. **Spring AI** gives you a well-run office. You ask `ChatClient` for what you need, and helpers (advisors) handle the filing, memory and tools, the Spring way.

Both assistants still read their instructions literally, so you'd better read them too.

Mechanism: the same agent twice

Spring AI

From the ChatClient and tools docs, as compiled and run in `SpringAiLab.java`:

```
@Tool(description = "Look up one order by ID. Use this for a known order number.")
String lookupOrder(
        @ToolParam(description = "Digits only, such as 7781") String orderId) {
```

```
String answer = ChatClient.create(model).prompt()
        .system("You are a support agent.")
        .user("Where is order 7781?")
        .tools(tools)
        .call()
        .content();
```

Concept	What it does
`ChatClient`	Fluent API. Auto-configured as a builder in Spring Boot
`@Tool` , `@ToolParam`	Generate the schema. All parameters are required by default
Advisors	Interceptors in a chain, for memory, RAG and logging
`ToolCallingAdvisor`	Runs the loop
`ToolContext`	Passes data such as a tenant ID to tools, **and isn't sent to the model**
`entity(Type)`	Structured output

`ToolContext` is the right home for identity. The model can't see or change it, unlike a tool parameter it fills in (AI09).

LangChain4j

From the AI Services and tools docs, as compiled and run in `LangChain4jLab.java`:

```
interface Support {
    @SystemMessage("You are a support agent.")
    String chat(@MemoryId String customerId, @UserMessage String message);
}
```

```
Support support = AiServices.builder(Support.class).chatModel(model).tools(tools)
        .chatMemoryProvider(id -> MessageWindowChatMemory.withMaxMessages(10))
        .build();
```

Concept	What it does
`AiServices`	Makes a proxy that implements your interface
`@Tool`, `@P`	Tool definitions
`@MemoryId`	Memory per user or conversation
`ChatMemoryProvider`	Creates memory per ID
`ContentRetriever`, `RetrievalAugmentor`	RAG
`Result<T>`	Adds token usage, sources and tool executions to a typed answer
Handlers	`toolExecutionErrorHandler`, `toolArgumentsErrorHandler`, `hallucinatedToolNameStrategy`

Don't call one `@MemoryId` concurrently, as the docs warn that it has no lock for this.

What actually gets sent

The recording model shows the loop running. Real output:

```
Spring AI:
  request 1: SYSTEM USER
  request 2: SYSTEM USER ASSISTANT TOOL
  tool ran 1 time, answer: Order 7781 was delivered.
LangChain4j:
  answer: Order 7781 was delivered.
  answer: You're welcome.
  request 1: SYSTEM USER
  request 2: SYSTEM USER AI TOOL_EXECUTION_RESULT
  request 3: SYSTEM USER AI TOOL_EXECUTION_RESULT AI USER
```

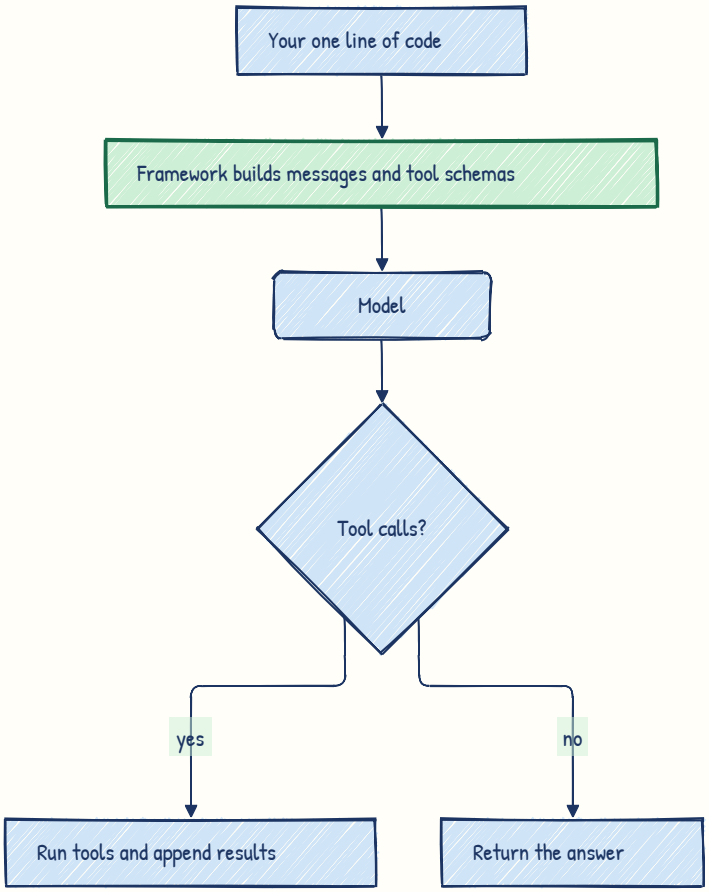


Figure 1. What the frameworks do between your one line of code and the answer. It's the same loop as AI06 L2, and all of it is hidden unless you log it.

Structured output: look at the prompt

```
Spring AI:   parsed: Refund[orderId=7781, amount=40.0]
             sent 667 characters for 17 typed
             starts: Refund order 7781 Your response should be in JSON format. Do not include a
LangChain4j: parsed: Refund[orderId=7781, amount=40.0]
             sent: "Extract the refund from: Please refund $40 on order 7781
                   You must answer strictly in the ...
```

Both default to asking through prompt text, which is the weaker approach (AI03 L3). Spring AI's docs offer `.useProviderStructuredOutput()`, to send the schema as an API constraint, and `.validateSchema()`, which retries with feedback. For LangChain4j, its docs recommend JSON mode for POJO extraction, with per-provider settings, and I haven't confirmed the Anthropic setting. **Prefer the API-level path where you can.**

Runaway loops

A scripted model that always calls a tool. Real output:

```
Spring AI:    40 tool runs, 41 requests, returned: Tool call limit (40) exceeded for
              tool 'lookupOrder'. No further calls to this tool are allowed in this turn.
LangChain4j:  stopped after 100 tool runs: Something is wrong, exceeded 100 tool calling
              round trips (maxToolCallingRoundTrips)
```

A difference that matters

LangChain4j raised an exception. Spring AI returned the limit message as if it were the answer, with no exception, even though its docs list `THROW` as the default. So here, a caller would show that text to a customer. I tested `ChatClient.create` without Spring Boot, and auto-configuration may differ. Test your own setup, and check for it (AI06 L2). A limit should report failure.

Compared

Facts below come from the docs I read and this module's labs:

Area	Spring AI 2.0.2	LangChain4j 1.23.0
Style	Fluent `ChatClient`, advisors	Interfaces as services
Fits	Spring Boot	Plain Java, Spring Boot, Quarkus
Spring Boot	2.0.x supports Boot 4.0 and 4.1	Starters for Boot 3 and Boot 4
Anthropic client	Official SDK, pinned at 2.7.2.0	Its own client
Jars with Anthropic	56, 65.9 MB	13, 5.5 MB
Caching	`AnthropicCacheStrategy`, for example `SYSTEM_AND_TOOLS`	`cacheSystemMessages`, `cacheTools`, `cacheAutomatically`
Tool limit, as observed	40 per tool, returned as text	100 round trips, exception
Error default	Message goes to the model	Message goes to the model
Identity for tools	`ToolContext`	Invocation parameters, `@MemoryId`
Multi-agent	Not read for this lesson	`langchain4j-agentic`, "experimental"
Backends	Bedrock and Vertex **not supported** for Anthropic module	Not confirmed

Spring Boot versions matter here. This course uses Boot 3 elsewhere, and Spring AI 2.0.x targets Boot 4. The docs also list 1.1.8 as stable, and I'd expect that line to be the Boot 3 route, but the page doesn't say so. Confirm before you choose. AI08 will settle this.

Current model pitfalls

Things in these docs to test on Opus 5.5 and Sonnet 5.5:

Feature	Docs show	On current models
Forced tool choice	`ToolChoiceAny`, `toolChoice(REQUIRED)`	**Returns a 400**
`temperature`	Spring AI's options table lists a default of `1.0`	Rejected, or non-default rejected. Check what's actually sent
Thinking	`thinkingEnabled(10000L)`, `thinkingBudgetTokens`	Budgets return a 400. Use adaptive
Effort	Spring AI: `.effort(...)`. LangChain4j: `customParameters`	Use it
Model names	Some old names in examples	Set yours explicitly

Both let you pass `claude-opus-5-5` as a string. **I haven't called the real API through either one,** so run a test call on your exact model.

Agents in LangChain4j

The `langchain4j-agentic` module has builders for sequence, loop, parallel and conditional workflows, and an LLM-driven supervisor. It maps onto AI06 L1 nicely. The docs say "the whole module has to be considered experimental and is subject to change", and I read only part of that long page. So use it with care, or write the orchestration yourself with virtual threads (AI06 L6).

Why the obvious fixes fall short

Tempting move	Why it disappoints	Better
"Tell the model not to repeat errors"	The secret has already reached the provider	Clean exception messages, or a handler
Trust the tool limit to report failure	Spring AI returned it as text, in this test	Check, and test
Portable `ChatOptions` everywhere	Caching and effort are provider-specific	Use provider options where they matter
`entity()` with defaults	Prompt-based JSON	Provider structured output
Identity as a tool parameter	The model fills it in, and can be tricked	`ToolContext`, or invocation parameters

How Anthropic and the field approach it

Point	Source
Spring AI's Anthropic module uses Anthropic's official Java SDK	Spring AI docs
Understand a framework's code	Building Effective Agents
"Risk-tier and destructive tools should typically be added per call, not as defaults"	Spring AI tools docs
Don't send raw exceptions in production	LangChain4j tools docs
`ToolContext` data isn't sent to the model	Spring AI tools docs

The production architecture

This is a recommended reference design, not a specific company's system. AI08 builds this.

Baseline: a controller calling `ChatClient` with default settings.

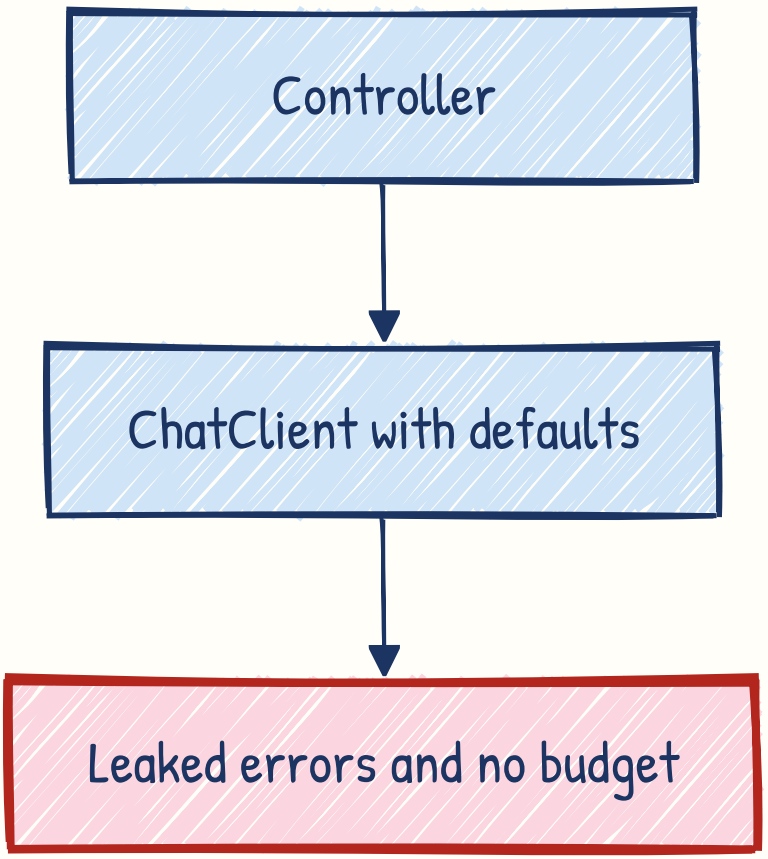


Figure 2. Baseline. A controller calls the framework directly, with default error handling and no spending limits.

Improved: a service layer, safe errors and limits. New problem: each team configures the framework differently, and framework types spread through services.

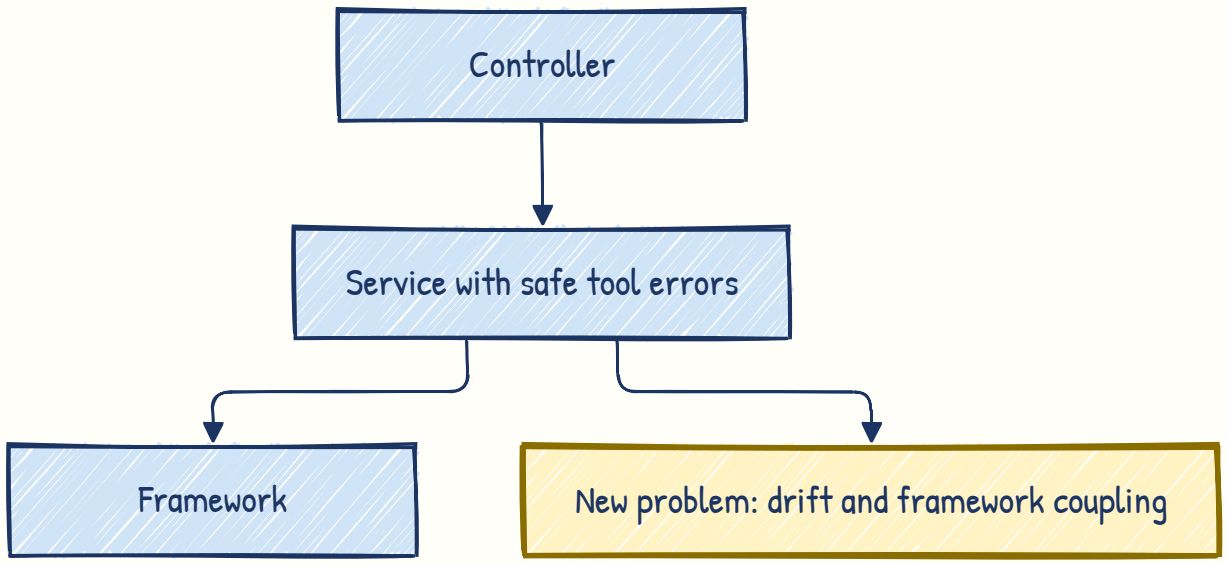


Figure 3. Improved. Services and handlers fix the defaults, but configuration differs per team, and framework types spread.

Production: a shared starter behind a port.

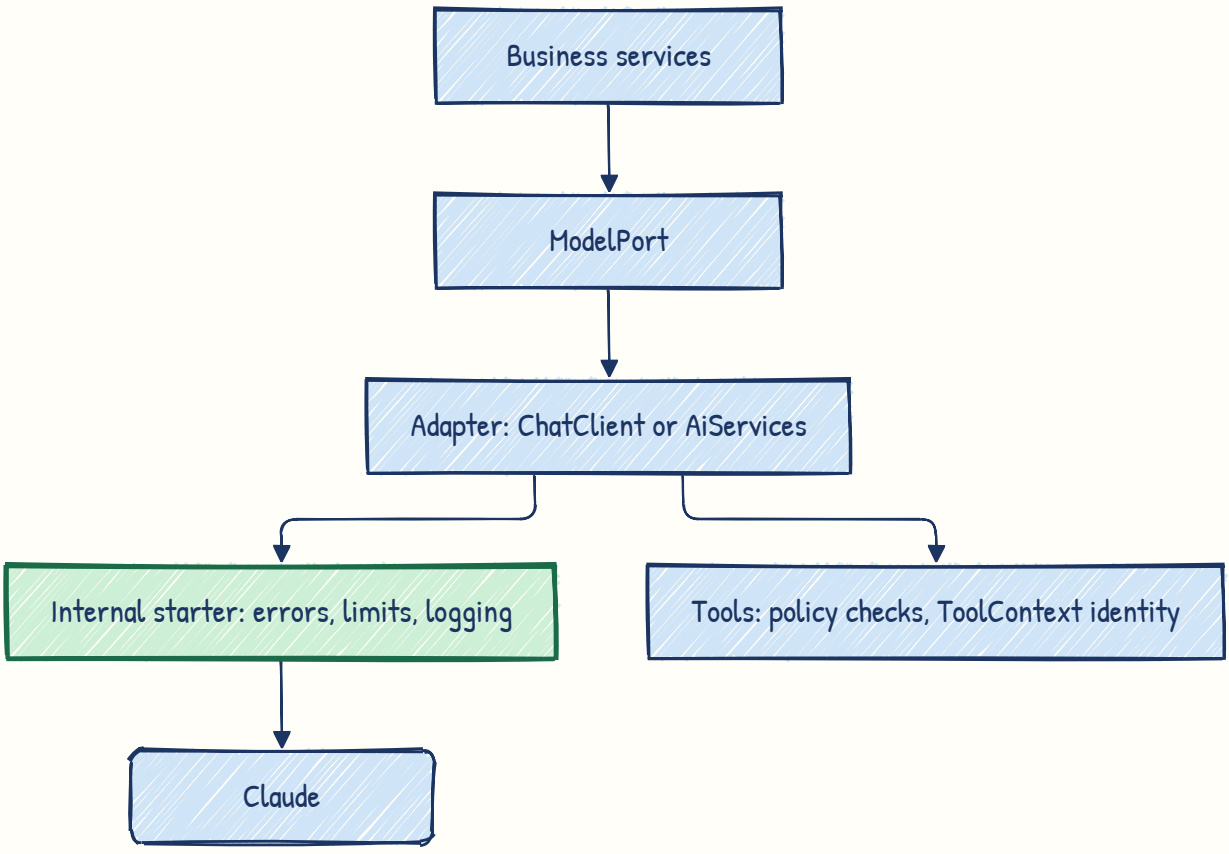


Figure 4. Production. An internal starter fixes defaults in one place. Services use a port, and policy runs in the tools, with real requests logged.

Component	Why it exists	How it communicates
Internal starter	One place for safe defaults	Spring auto-configuration
Error handler	Keeps secrets out, and categorizes errors (AI06 L5)	`ToolExecutionExceptionProcessor`, or a LangChain4j handler
Policy in tools	Guarantees (AI06 L8)	Runs before the backend call
Request logging	See what's sent	`SimpleLoggerAdvisor` or listeners, with redaction

Failures

Failure	Containment
Secret in an exception	Handler, plus exception hygiene, plus a test
Limit message reaches a user	Detect it, and treat as failure
New model rejects a default	Nightly test on the actual model
Framework upgrade changes defaults	Pin versions, and diff recorded requests
Concurrent memory corruption	One request per memory ID, or a lock

Trade-offs

Decision	Gain	Cost
Spring AI	Native Spring feel, observability, official SDK	Heavier, tied to Boot versions, pinned SDK
LangChain4j	Light, framework-neutral, helpful handlers	Own client may lag on Anthropic features
Raw `anthropic-java`	Newest API, clearest	You write memory, schemas and loop, or use the beta runner

Choosing: Spring Boot shop with vector stores: Spring AI. Plain Java or Quarkus, or wanting a small footprint: LangChain4j. A single agent wanting the latest features: the SDK. Verify with this module's scripted tests either way.

Common mistakes

1. **Leaving default error handling.** Exceptions reach the model.
2. **Assuming limits raise errors.** Test it.
3. **Using `getDefaultOptions()` in a Spring AI 2.0 fake.** It's deprecated, and the tool loop silently didn't run until I overrode `getOptions()`.
4. **Using `entity()` without provider structured output.** Prompt-based JSON is weaker.
5. **Passing tenant IDs as tool parameters.** Use `ToolContext`.
6. **Using forced tool choice on Opus 5.5.** It returns a 400.
7. **Assuming a Boot 3 app can take Spring AI 2.0.** Check compatibility.
8. **Calling one memory ID concurrently.** LangChain4j has no lock for it.

What to say in an interview

"For a Spring shop, I'd consider Spring AI, and for plain Java, LangChain4j. I ran both against a scripted model: both send tool exception messages to the model by default, so I'd set a handler and keep messages clean. LangChain4j raised an exception at its loop limit, while Spring AI returned the limit message as text in my test. So I'd put defaults in an internal starter, and test on our exact model."

Certification lens

How this lesson maps to CCAR-F

Spring AI and LangChain4j aren't tested, and neither is any language-specific implementation. The underlying ideas are: tool loops (TS 1.1), tool errors (TS 2.2), and `tool_choice` (TS 2.3).

These are original practice questions for understanding, not real or leaked exam questions.

Q Q1 · Debugging · Hard. A LangChain4j tool throws `new RuntimeException("JDBC failed: jdbc:mysql://prod-db:3306 user=app")`. What reaches the model by default? Give two fixes.

A The whole message, including host and user. (1) Set `ToolExecutionErrorHandler.failInvocationUnlessVisibleToLlm()`, and throw only exceptions implementing `ToolErrorVisibleToLlm` with safe text. (2) Catch inside the tool, log the detail, and return a safe, categorized message (AI06 L5).

Q Q2 · Security · Medium. A tool takes `tenantId` as a parameter that the model fills in. What's the risk, and how do you fix it in Spring AI?

A The model chooses it, so an injected instruction could read another tenant's data. Pass it through `ToolContext`, which isn't sent to the model, set by your code from the authenticated request. The tool reads it from there.

Q Q3 · Best decision · Medium. A Spring Boot 3 team wants to adopt Spring AI today. What should they check first, and what's the alternative if it doesn't fit?

A Compatibility: Spring AI 2.0.x targets Boot 4.0 and 4.1, so they should confirm which 1.x line suits Boot 3, and whether it supports their model and features. Alternatives are upgrading Boot, using LangChain4j's Boot 3 starter, or calling the official SDK directly.

Module Revision

Revision - AI07

→ A framework trades code you'd write for behavior you can't see.

→ Decide by who owns the harness and who owns the deployment.

→ Keep frameworks behind your own port, and log real requests.

→ Anthropic: start with the API, and understand any framework's code.

→ LangGraph: state, nodes, edges, checkpoints, interrupts.

→ A resumed node restarts from its first line, so keep side effects after it.

→ LangChain's agents are built on LangGraph. LangGraph doesn't need LangChain.

→ LlamaIndex centers on data: load, index, store, query, evaluate.

→ Two-line demos hide chunking, prompts and search choices.

→ CrewAI: Flows control, Crews do team steps.

→ AutoGen's README says archived, so check repositories for status.

→ Script a stuck model and a failing tool to learn a framework's defaults.

→ Spring AI and LangChain4j both send tool exception messages to the model by default.

→ LangChain4j throws at its loop limit. Spring AI returned text in my test.

→ Prefer provider structured output over prompt-based JSON.

→ Identity belongs in ToolContext, not in model-filled parameters.

→ Test on your exact model, since defaults can return a 400.

Self-assessment

#	Without notes, can you answer?
1	What two questions separate the options?
2	Compare Messages API, Tool Runner, Agent SDK and Managed Agents.
3	How can a Java team drive the Agent SDK's loop?
4	What do frameworks give, and what do they cost?
5	What's the port pattern for, and what goes in the adapter?
6	What's Anthropic's advice on frameworks?
7	How do LangChain, LangGraph, Deep Agents and LangSmith relate?
8	What's a checkpointer? A thread?
9	What happens to a node on resume? What must you do about it?
10	Give three `interrupt()` pitfalls.
11	When is a graph overkill?
12	Name LlamaIndex's five stages.
13	What defaults did the lab reveal? What would you change?
14	How do Flows and Crews relate?
15	Why must you check a project's repository as well as its docs?
16	Give the evaluation checklist.
17	How do you find out what a framework really sends?
18	Compare Spring AI's and LangChain4j's programming styles.
19	What reaches the model when a tool throws? How do you control it?
20	How did each behave on a runaway loop?
21	Why prefer provider structured output?

#	Without notes, can you answer?
22	Where should tenant identity go, and why?
23	What should you test on Opus 5.5 before trusting a framework?
24	How would you choose between Spring AI, LangChain4j and the raw SDK?

Where this module is used

Idea	Where it returns
Port and adapter, scripted models	AI08 L1–L3
Spring AI or LangChain4j in a service	AI08
Tool error handling, tenant identity	AI08 L3, AI09
Request logging, nightly model tests	AI10
Agent SDK	AI12

Working space